Assignment – 8

Auto Scaling Web Servers

1. Objective -

The objective of this assignment was to deploy a scalable web server infrastructure on AWS using **Amazon EC2 Auto Scaling**, **Application Load Balancer (ALB)**, **Target Groups**, **Launch Templates**, **Dynamic Scaling Policies**, and **AWS Systems Manager**.

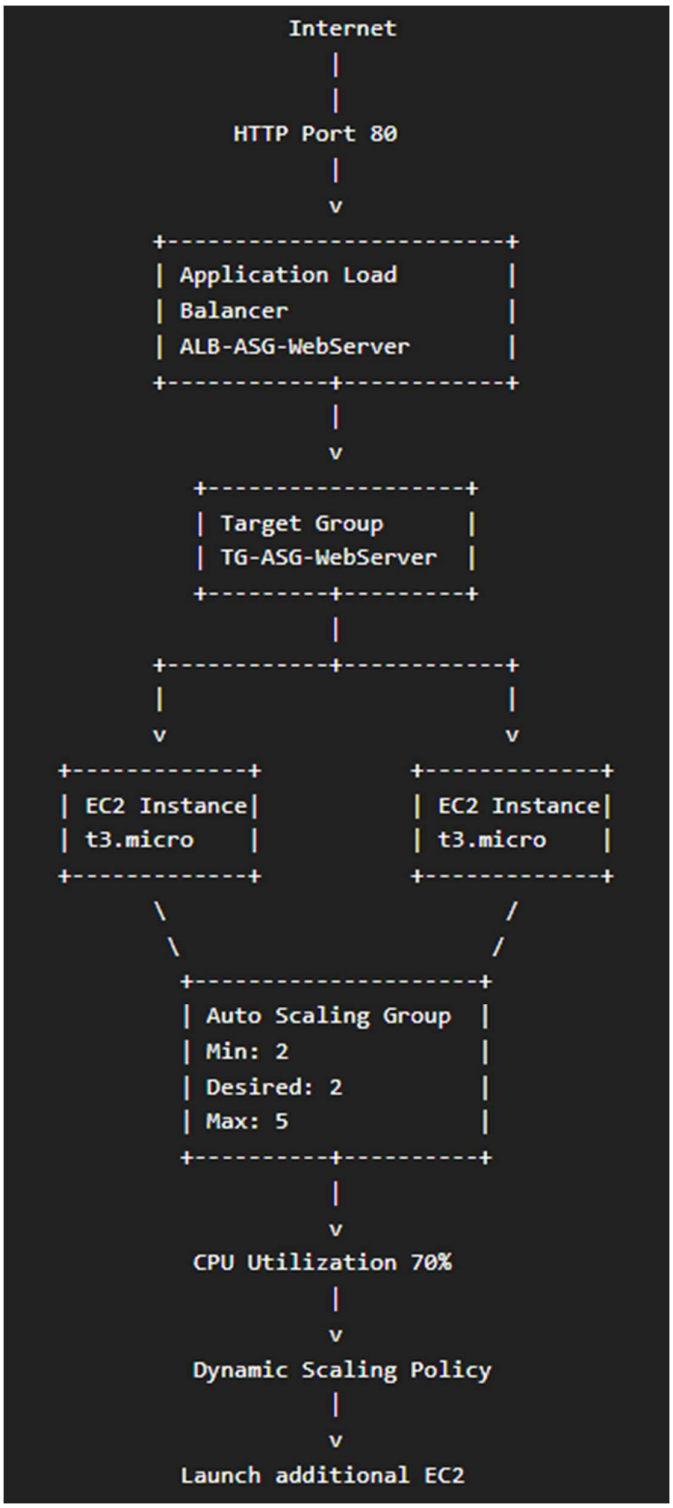
The implementation was designed to:

- Automatically launch EC2 web servers.
- Distribute incoming HTTP traffic using an Application Load Balancer.
- Maintain a minimum and maximum number of EC2 instances.
- Automatically increase or decrease the number of instances according to CPU utilization.
- Monitor the scaling activity through AWS CloudWatch and Auto Scaling activity logs.
- Use AWS Systems Manager to execute commands on managed EC2 instances.
- Test the automatic scaling mechanism by generating CPU load.

2. AWS Services Used –

AWS Service	Purpose
Amazon EC2	Hosts the web server instances
EC2 Launch Template	Defines the configuration for new EC2 instances
Security Group	Controls inbound and outbound traffic
Application Load Balancer	Distributes HTTP traffic
Target Group	Registers EC2 instances with the load balancer
EC2 Auto Scaling	Automatically adjusts the number of EC2 instances
CloudWatch	Provides CPU utilization monitoring and scaling metrics
AWS Systems Manager	Executes commands on EC2 instances remotely
IAM	Provides required permissions through IAM roles

3. Architecture –



Architecture explanation

The Application Load Balancer receives HTTP requests from users and forwards them to healthy EC2 instances registered in the target group.

The EC2 instances are managed by the Auto Scaling Group. The Auto Scaling Group maintains the configured capacity between **2 and 5 instances**.

A target tracking scaling policy maintains average CPU utilization around **70%**. When CPU utilization increases significantly, the Auto Scaling Group launches additional instances.

4. Security Group Configuration -

A dedicated security group named:

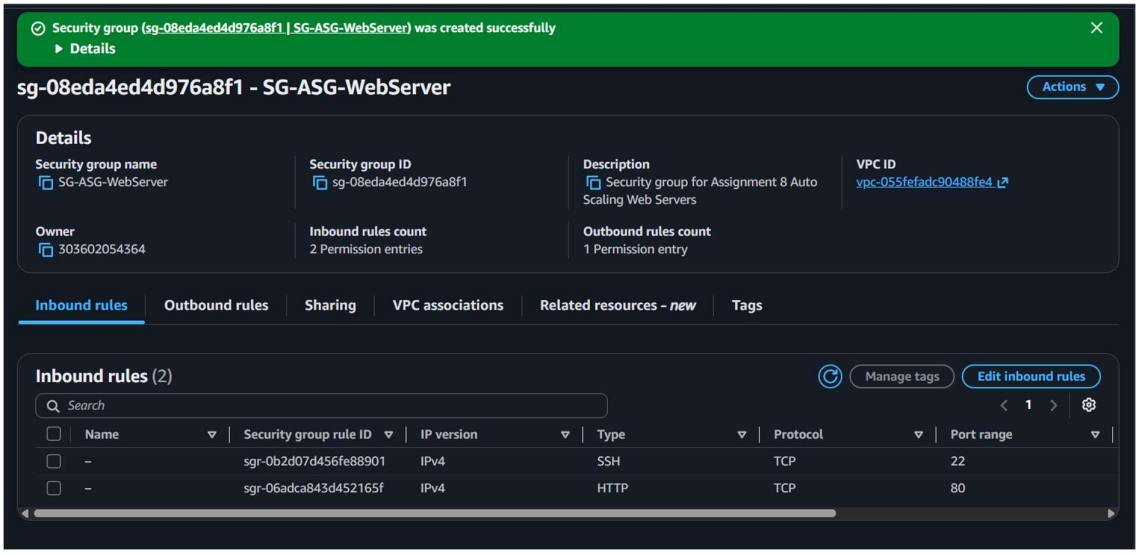
SG-ASG-WebServer

was created for the web server infrastructure.

The security group contains:

- **SSH - TCP 22**
- **HTTP - TCP 80**
- Outbound traffic required for the instances.

Figure 1: Security Group SG-ASG-WebServer configuration



Explanation

The security group was configured to allow SSH access for administration and HTTP traffic for accessing the web server. This security group was associated with the EC2 instances launched through the Auto Scaling Group.

5. IAM Role for Systems Manager –

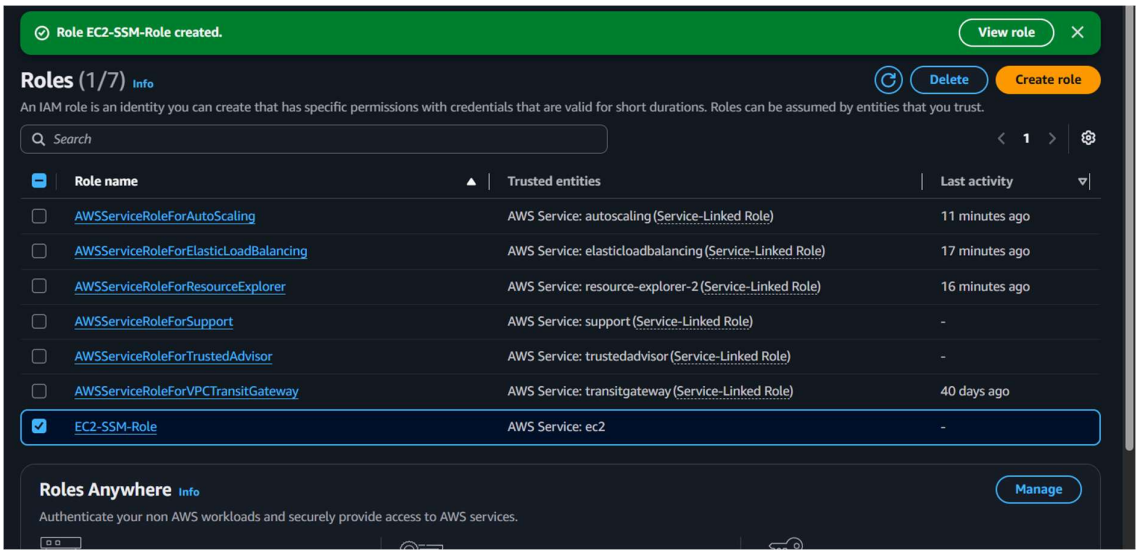
An IAM role named:

EC2-SSM-Role

was created and used for EC2 Systems Manager functionality.

The role allowed the EC2 instances to communicate with AWS Systems Manager and appear as managed nodes in Fleet Manager.

Figure 2: EC2-SSM-Role created successfully



Explanation

The EC2-SSM-Role IAM role was created for the EC2 instances. It enabled Systems Manager functionality, allowing commands to be executed remotely without relying entirely on SSH.

6. Launch Template Creation –

A launch template named:

LT-ASG-WebServer

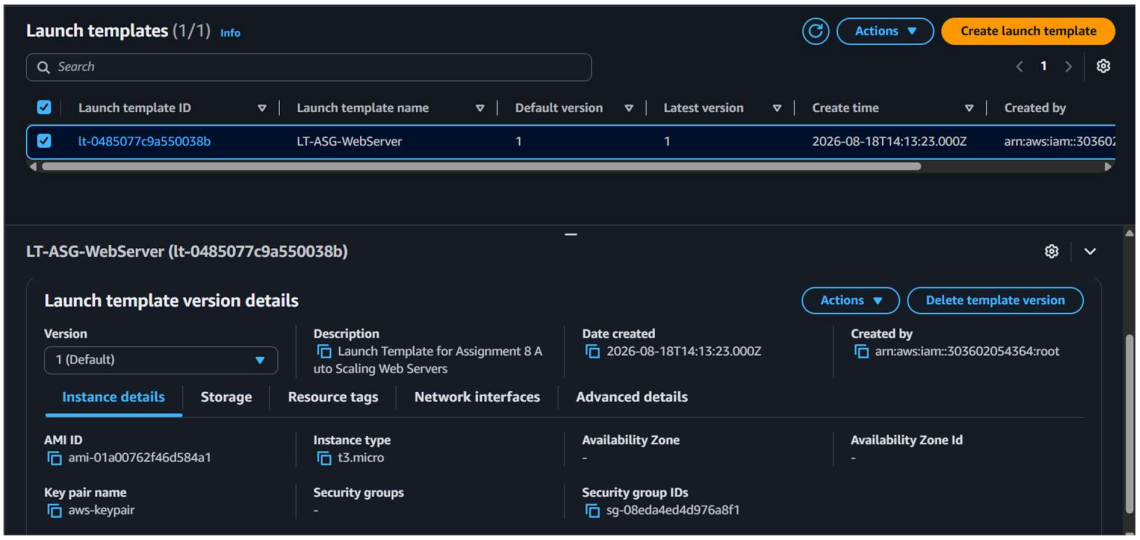
was created.

The launch template defined the configuration used whenever the Auto Scaling Group launches a new EC2 instance.

Configuration

- **AMI:** Ubuntu
- **Instance Type:** t3.micro
- **Key Pair:** aws-keypair
- **Security Group:** SG-ASG-WebServer
- Launch template version: **1**

Figure 3: LT-ASG-WebServer Launch Template



Explanation

The launch template provides a standard configuration for all EC2 instances launched by the Auto Scaling Group. This ensures that newly launched instances use the same operating system, instance type, security group and other configuration parameters.

7. Target Group Creation -

A target group named:

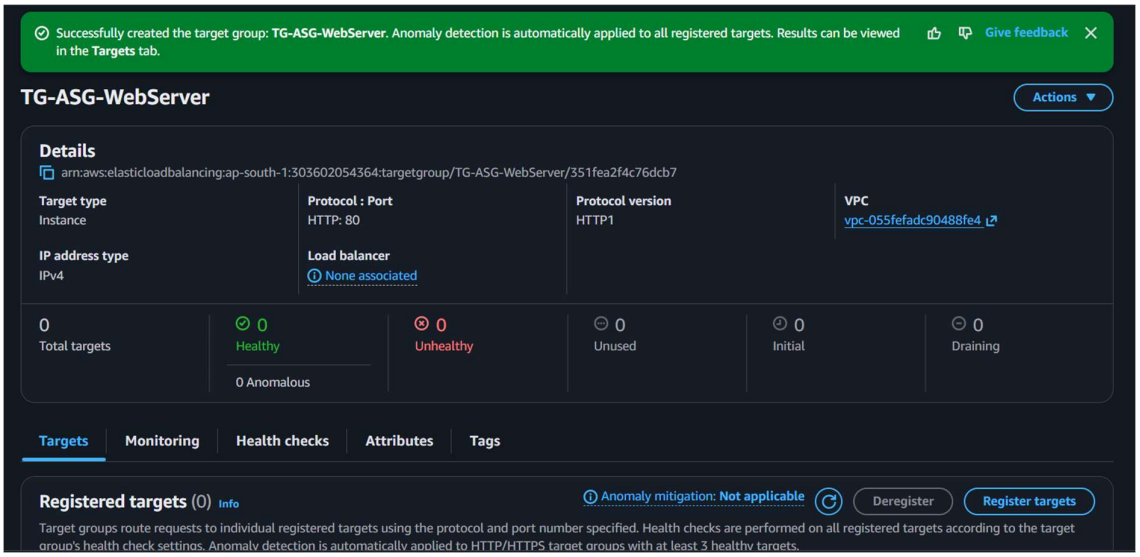
TG-ASG-WebServer

was created.

Configuration

- **Target type:** Instance
- **Protocol:** HTTP
- **Port:** 80
- **Protocol version:** HTTP/1
- **VPC:** Custom VPC

Figure 4: TG-ASG-WebServer Target Group



Explanation

The target group acts as a collection of EC2 instances that receive traffic from the Application Load Balancer. Health checks are used to determine whether registered instances are healthy and capable of receiving traffic.

8. Application Load Balancer Creation -

An Application Load Balancer named:

ALB-ASG-WebServer

was created.

Configuration

- **Type:** Application Load Balancer
- **Scheme:** Internet-facing
- **IP address type:** IPv4
- **Availability Zones:** ap-south-1a and ap-south-1b
- **Listener:** HTTP : 80
- **Target Group:** TG-ASG-WebServer

Figure 5: Application Load Balancer ALB-ASG-WebServer

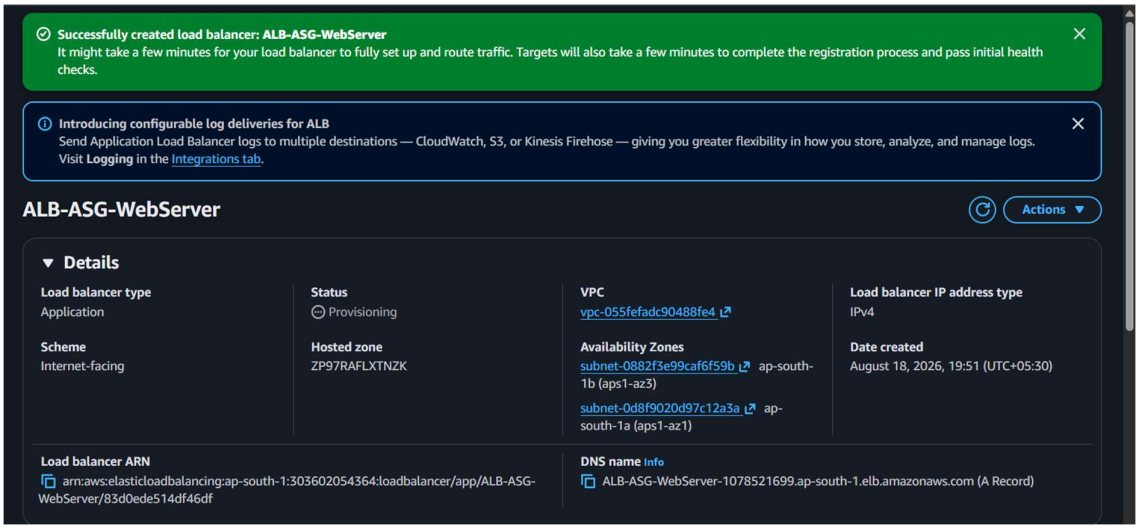
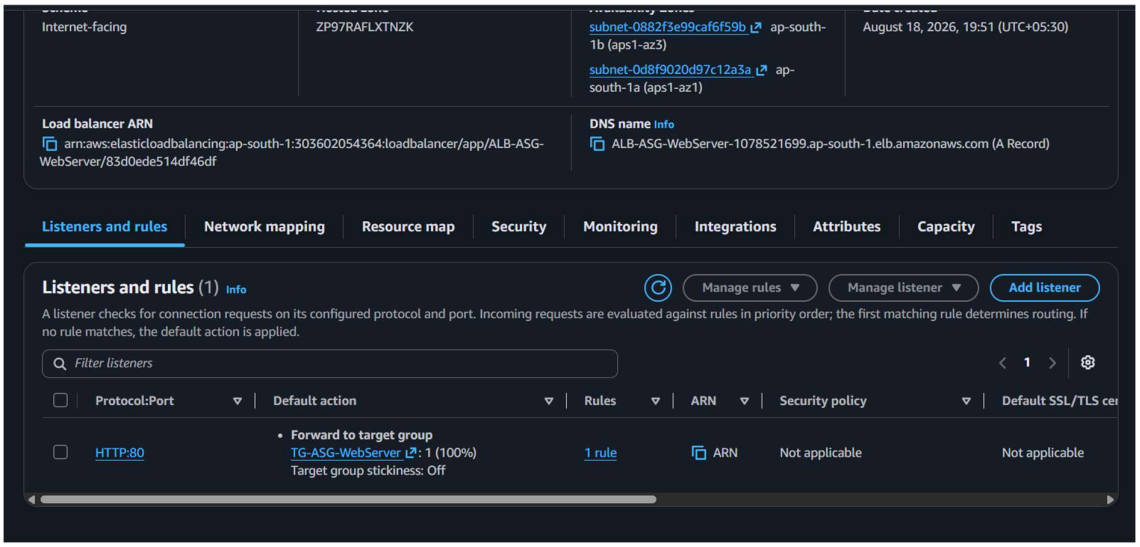


Figure 6: ALB listener forwarding HTTP traffic to TG-ASG-WebServer



Explanation

The Application Load Balancer was configured as an internet-facing load balancer. Its HTTP listener on port 80 forwards incoming requests to the TG-ASG-WebServer target group.

This allows traffic to be distributed among multiple EC2 instances instead of depending on a single server.

9. Auto Scaling Group Creation –

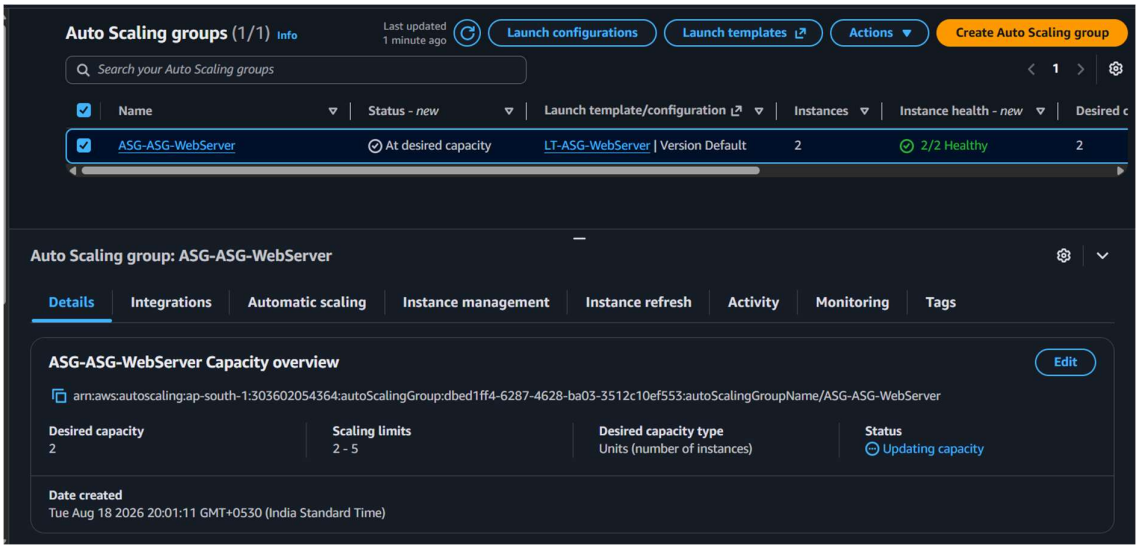
An Auto Scaling Group named:

ASG-ASG-WebServer

was created using the LT-ASG-WebServer launch template.

Parameter	Value
Minimum capacity	2
Desired capacity	2
Maximum capacity	5
Instance type	t3.micro
Availability Zones	2
Launch Template	LT-ASG-WebServer

Figure 7: ASG-ASG-WebServer Auto Scaling Group



Explanation

The Auto Scaling Group was configured to maintain at least two EC2 instances and scale up to a maximum of five instances according to workload requirements.

The initial desired capacity was two instances.

10. Dynamic Scaling Policy -

A target tracking dynamic scaling policy named:

CPU-70-Scaling

was created.

Policy configuration

- **Policy type:** Target tracking scaling
- **Target metric:** Average CPU utilization
- **Target value:** 70%
- **Scale-in:** Enabled
- **Instance warm-up:** 300 seconds
- **Action:** Add or remove capacity as required

Figure 8: Dynamic Scaling Policy configuration

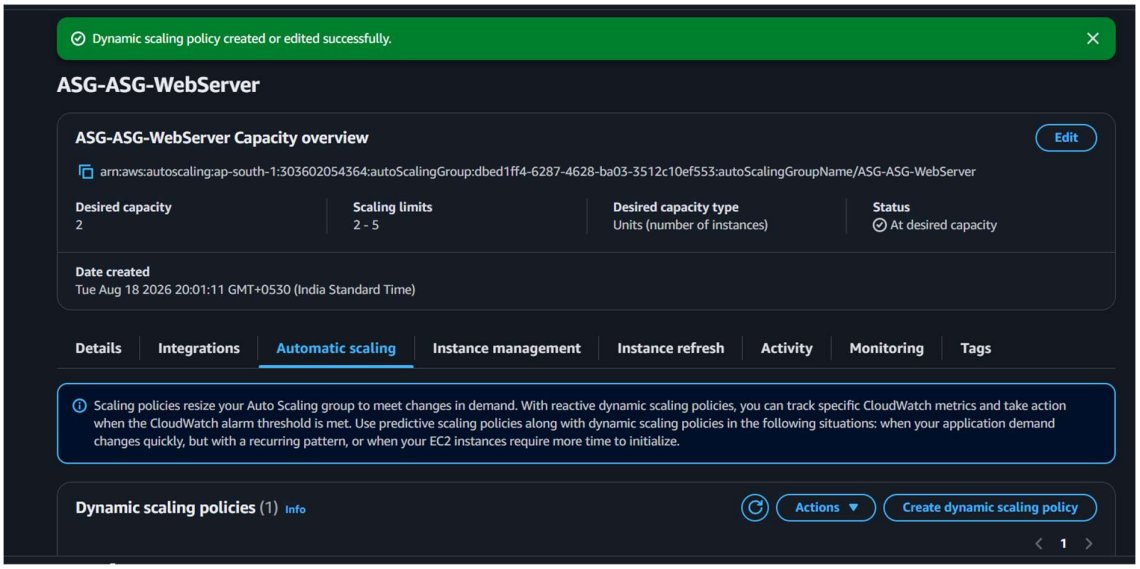
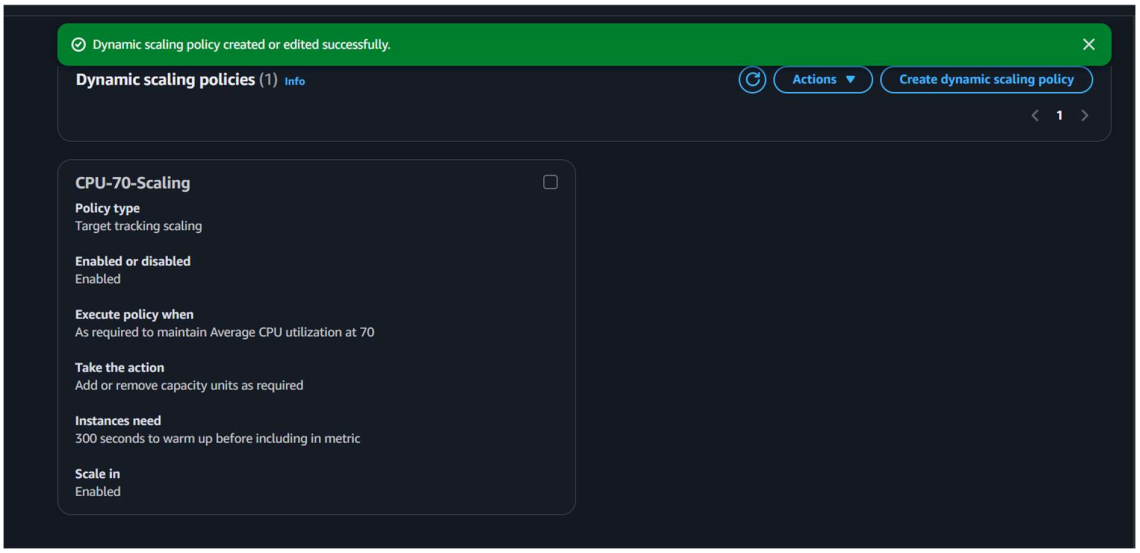


Figure 9: CPU-70-Scaling policy details



Explanation

The target tracking policy continuously monitors the average CPU utilization of the Auto Scaling Group.

When the CPU utilization exceeds the target level, Auto Scaling increases the desired capacity by launching additional instances.

When the workload decreases, Auto Scaling can reduce the number of instances while respecting the minimum capacity.

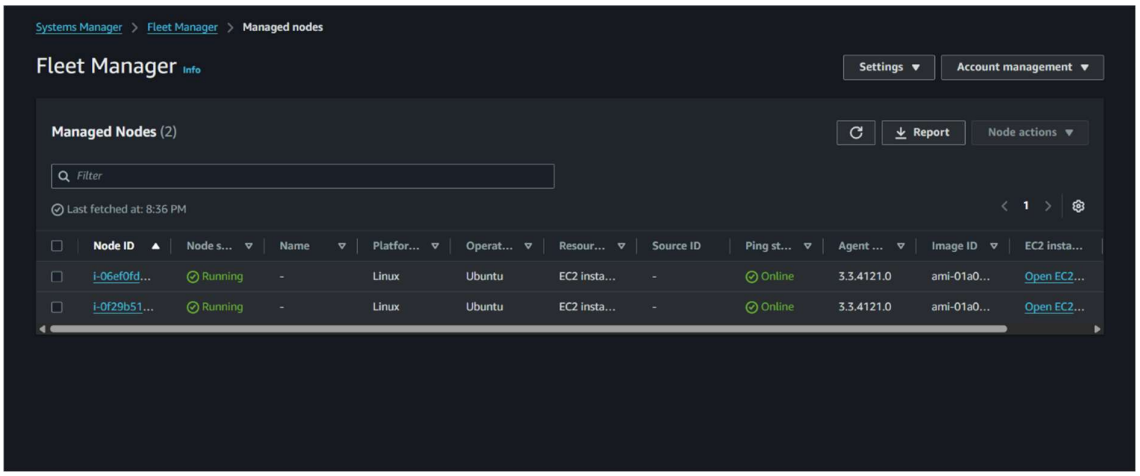
11. Systems Manager Managed Nodes -

The EC2 instances were successfully registered with AWS Systems Manager.

The Fleet Manager page showed **2 managed nodes**, both:

- Running
- Linux
- Ubuntu
- Online
- SSM Agent version 3.3.4121.0

Figure 10: EC2 instances registered as Systems Manager Managed Nodes



Explanation

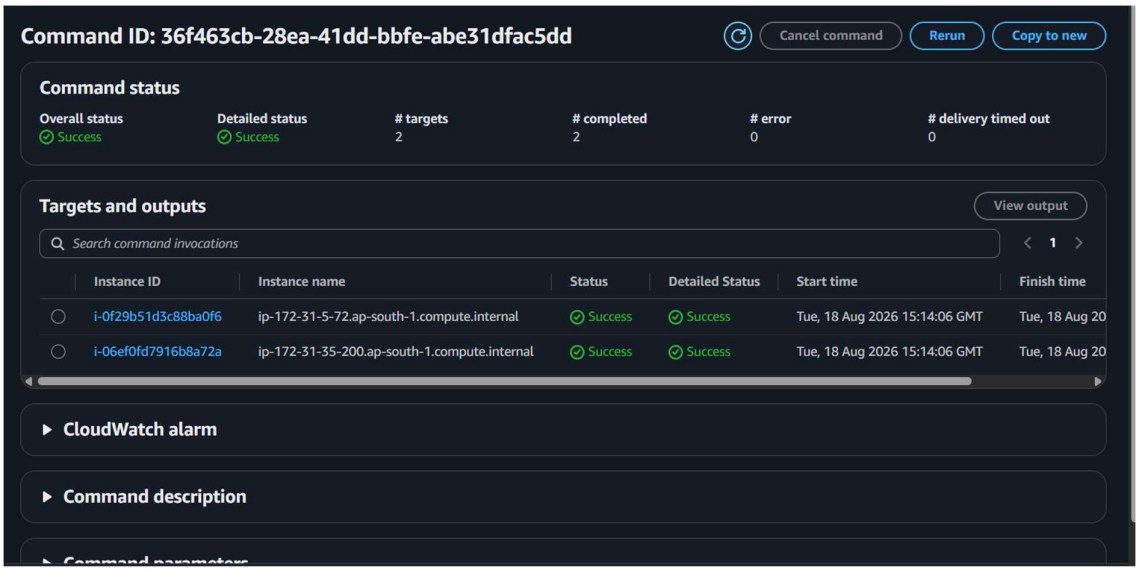
The managed nodes confirmed that the EC2 instances were successfully connected to AWS Systems Manager. This allowed remote commands to be executed on the instances.

12. Remote Command Execution

AWS Systems Manager Run Command was used to execute a command on the two EC2 instances.

The command completed successfully on both targets.

Figure 11: Successful Systems Manager Run Command execution



Evidence shown

- **2 targets**
- **2 completed**
- **0 errors**
- **0 delivery timeouts**
- Both instances: **Success**

Explanation

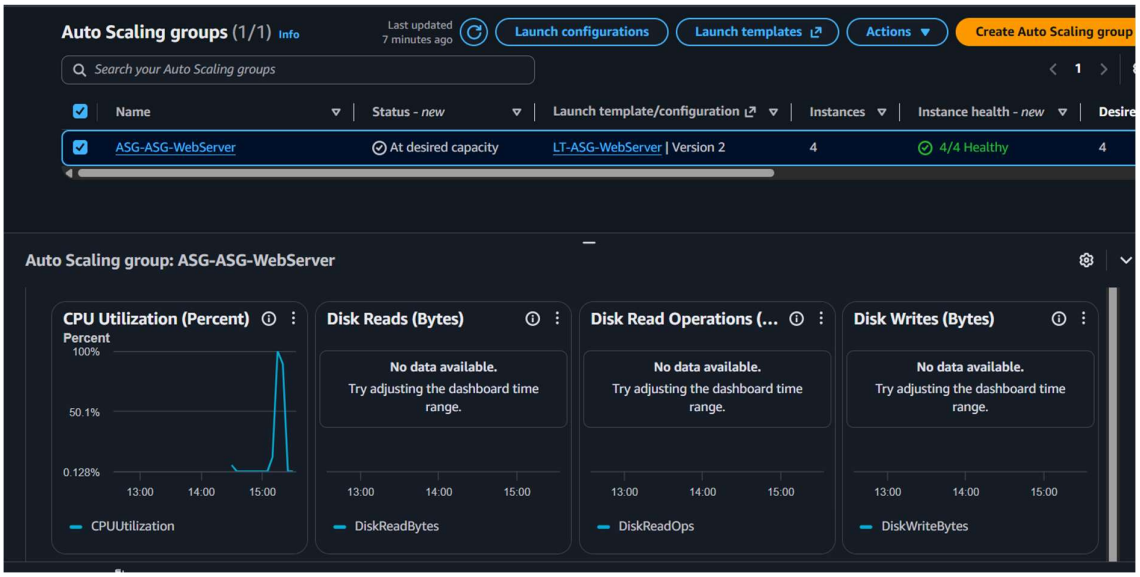
The successful command execution verified that Systems Manager was correctly configured and that commands could be remotely executed on the EC2 instances.

13. CPU Load Generation and Monitoring -

To test the Auto Scaling mechanism, CPU load was generated on the EC2 instances.

The CloudWatch monitoring graph showed a significant increase in CPU utilization, reaching approximately **100%** during the test.

Figure 12: CPU utilization spike during load testing



Explanation

During the stress test, CPU utilization increased sharply. The CPU graph shows a spike reaching approximately 100%, which triggered the configured target tracking scaling policy.

This provided the workload required to demonstrate automatic scaling.

14. Auto Scaling Activity -

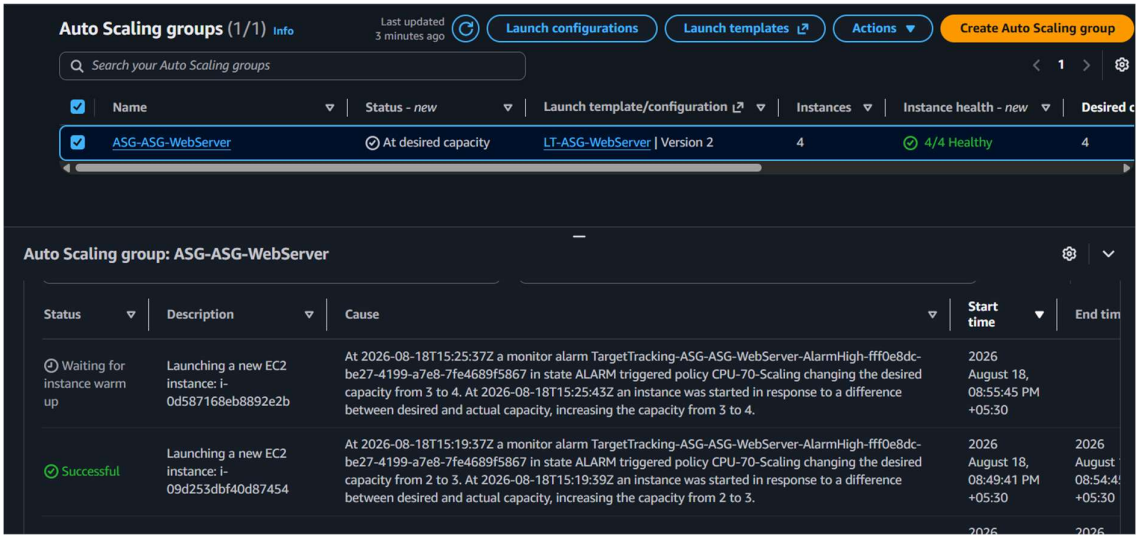
The Auto Scaling activity history confirmed that the scaling policy responded to the increased CPU utilization.

The activity log showed that the target tracking alarm triggered the CPU-70-Scaling policy and increased the desired capacity.

For example, the activity history showed:

Desired capacity: 2 → 3 → 4

Figure 13: Auto Scaling activity showing automatic instance launches



Explanation

The activity history provides direct evidence that the scaling policy worked. When the CloudWatch alarm entered the ALARM state because of high CPU utilization, the Auto Scaling Group increased its desired capacity and launched new EC2 instances.

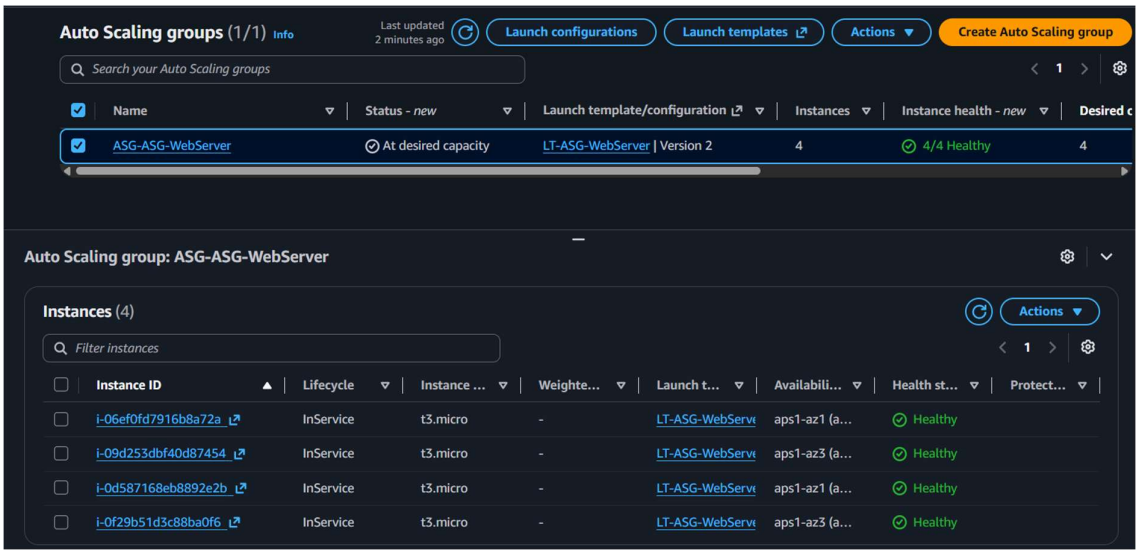
15. Auto Scaling Result -

After scaling, the Auto Scaling Group reached:

4 EC2 instances

All four instances were shown as healthy and in service.

Figure 14: Auto Scaling Group with 4 healthy instances



Explanation

The Auto Scaling Group successfully increased the number of EC2 instances from the initial capacity of 2 to 4 instances in response to increased CPU utilization.

The instances were distributed across the configured Availability Zones and reported healthy.

This confirms that the automatic scale-out mechanism was functioning correctly.

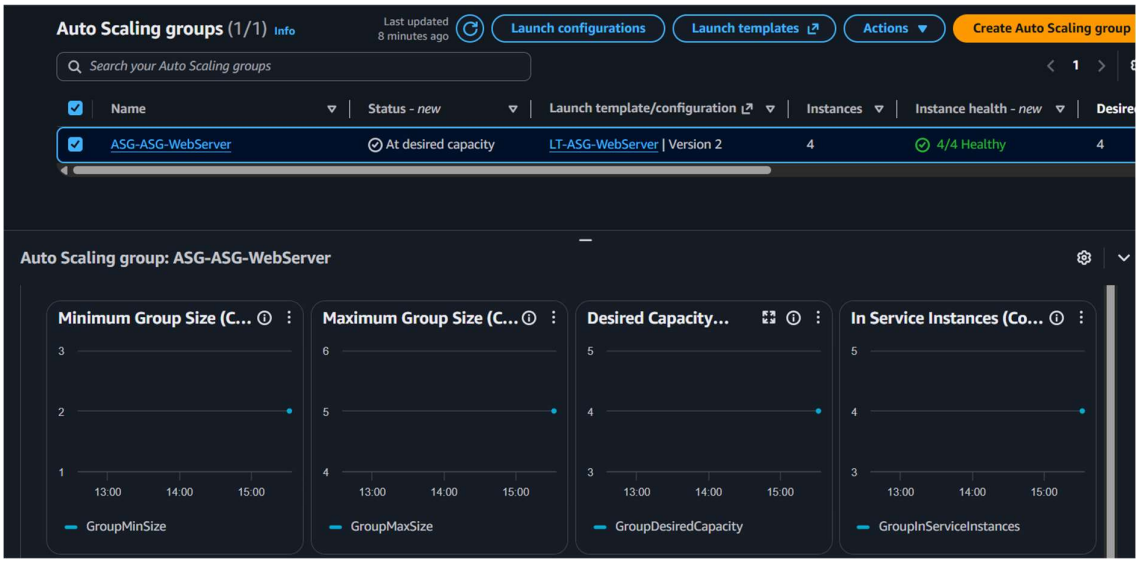
16. Auto Scaling Monitoring -

The Auto Scaling monitoring dashboard displayed metrics including:

- Minimum Group Size
- Maximum Group Size
- Desired Capacity
- In-Service Instances

The graphs demonstrated the capacity changes during the test.

Figure 15: Auto Scaling capacity monitoring



Explanation

The monitoring graphs provide a visual representation of the Auto Scaling Group's capacity configuration and scaling behavior.

17. Final Implementation Result -

The implementation successfully demonstrated an automatically scalable web server architecture.

Initial state

Minimum capacity = 2

Desired capacity = 2

Maximum capacity = 5

During CPU stress

High CPU utilization

↓

CloudWatch alarm

↓

CPU-70-Scaling policy

↓

Auto Scaling triggered

↓

New EC2 instances launched

Final observed state

EC2 Instances = 4

Healthy Instances = 4

Desired Capacity = 4

The scaling activity log confirmed the capacity increase from **2 → 3 → 4**.

18. Challenges Faced and Solutions -

Challenge 1: Configuring Systems Manager

Problem: EC2 instances needed to appear as managed nodes in Systems Manager.

Solution: An IAM role named EC2-SSM-Role was created and associated with the EC2 instances. After configuration, the instances appeared as online managed nodes in Fleet Manager.

Challenge 2: Verifying Remote Command Execution

Problem: It was necessary to verify that Systems Manager could communicate with the instances.

Solution: AWS Systems Manager Run Command was executed against both managed instances. Both targets completed successfully with zero errors.

Challenge 3: Triggering Auto Scaling

Problem: Normal CPU utilization was not high enough to trigger the 70% target tracking policy.

Solution: CPU load was generated on the EC2 instances. This caused CPU utilization to increase significantly and triggered the scaling policy.

Challenge 4: Verifying Scaling Behavior

Problem: It was necessary to verify that the policy actually launched additional instances rather than simply changing the desired capacity.

Solution: The Auto Scaling activity history and instance management page were checked. The activity log confirmed instance launches and the Auto Scaling Group eventually showed four healthy instances.

---

19. Security Considerations -

The following security practices were implemented:

- A dedicated security group was created for the web server.
- HTTP traffic was allowed on port 80.
- SSH access was configured on port 22.
- EC2 Systems Manager was used for remote command execution.
- IAM role-based access was used instead of embedding AWS credentials on the instances.
- Auto Scaling was used to maintain availability during increased workload.

---

20. Resource Cleanup -

After completing the testing and taking the required screenshots, the AWS resources created for the assignment were deleted to prevent unnecessary AWS charges.

Resources cleaned up included:

- Auto Scaling Group
- EC2 instances
- Application Load Balancer
- Target Group
- Launch Template
- Security Group
- EC2-SSM-Role
- Related scaling resources

21. Conclusion -

The assignment successfully demonstrated the implementation of a scalable and highly available web server environment on AWS.

An Application Load Balancer was configured to distribute HTTP traffic through a target group, while an Auto Scaling Group managed the EC2 web servers. A target tracking policy maintained CPU utilization around 70% and automatically increased the number of EC2 instances when CPU utilization became high.

During testing, CPU utilization increased significantly, triggering the scaling policy. The Auto Scaling Group successfully increased capacity from **2 to 3 and then to 4 instances**, with all four instances becoming healthy.

AWS Systems Manager was also successfully configured and used to execute commands on the managed EC2 instances.

Therefore, the assignment successfully demonstrated **load balancing, automatic scaling, monitoring, remote management, and cloud resource management using AWS.**